

Module 4

Pointers: Fundamentals, Pointer Declarations, Operations On Pointers, Passing Pointers To A Function, Pointers And One Dimensional Arrays, Pointers And Multidimensional Arrays, Array Of Pointers, Pointers And Strings, Multiple Indirection (Integers, Strings, Pointers To Pointers), Dynamic Memory Allocation (Malloc, Calloc, Realloc, Free). Structures And Unions: Defining A Structure, Processing A Structure, User Defined Data Types, Structure And Pointers, Passing Structure To Function, Self referential Structures, And Union

Pointer

A **pointer** is a derived data type in C that stores the **memory address of another variable**.

Instead of holding a direct value, it holds the address where the value is stored in memory. It is the backbone of low-level memory manipulation in C.

- A pointer is declared using its data type followed by an asterisk (*) and the pointer name, indicating the type of data it can store the address of.
- A pointer variable stores the memory address of another variable, while the dereference operator (*) is used to access the value stored at that address.
- The * symbol has two purposes in C: it is used to declare a pointer and to dereference a pointer to access the value it points to.

A pointer stores the **address of that memory location**.

```
int age = 25;
int *ptr;
ptr = &age;
printf("%d", *ptr);
```

Here:

- age stores the value 25
- &age gives the memory address of age
- ptr stores the address of age
- *ptr accesses the value stored at that address

Advantages of Pointers

1. Pointers can be used to return multiple values from a function via function arguments.
2. Pointers permit references to functions and thereby facilitate passing of functions as arguments to other functions.
3. The use of pointer arrays to character strings results in saving of data storage space in memory.
4. Pointers allow C to support dynamic memory management.

5. Pointers provide an efficient tool for manipulating dynamic data structures such as structures, linked lists, queues, stacks and trees.

The computer's memory is a sequential collection of storage cells. Each cell, commonly known as a byte, has a number called an address associated with it. The addresses are numbered consecutively, starting from zero. The last address depends on the memory size. A computer system having 64 K memory will have its last address as 65,535.

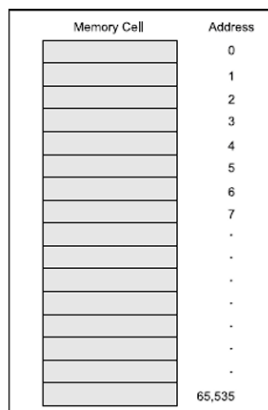


Fig. 12.1 Memory organisation

Whenever we declare a variable, the system allocates an appropriate location in memory to hold the value of the variable. Since every byte has a unique address, this location will have its own address number.

Consider the following statement:

```
int quantity = 179;
```

This statement instructs the system to find a location for the integer variable quantity and put the value 179 in that location. Let us assume that the system has chosen the address location 5000 for quantity.

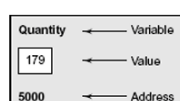


Fig. 12.2 Representation of a variable
int quantity = 179;

The address of a variable is the address of the first byte occupied by that variable.

During execution of the program, the system associates the name quantity with its memory address. We may access the value 179 by using either the name quantity or its address.

Since memory addresses are simply numbers, they can be assigned to variables and stored in memory. Such variables that hold memory addresses are called pointer variables.

A pointer variable is therefore a variable that contains an address, which is the location of another variable in memory.

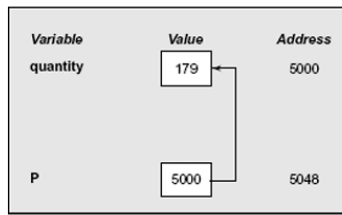


Fig. 12.3 Pointer variable

Suppose we assign the address of quantity to a variable p.

```
int quantity = 179;
```

```
int *p;
```

```
p = &quantity;
```

If the address of quantity is 5000, then:

```
quantity = 179
```

```
p = 5000
```

Since the value of the variable p is the address of the variable quantity, we may access the value of quantity by using p. Therefore, we say that the variable p points to the variable quantity. Thus, p is called a pointer.

The actual address values may be different every time the program runs. What is important is the relationship between the variables p and quantity.

Syntax

```
data_type *pointer_name;
```

Initialize the Pointer

A pointer is initialized by assigning it the address of a variable using the address operator (&)

- Initializing a pointer ensures it points to a valid memory location before use.
- You can also initialize a pointer to NULL if it doesn't point to any variable yet: `int *ptr = NULL;`

Syntax

```
pointer_name = &variable;
```

Operations on Pointers

A pointer is a variable that stores the address of another variable. Various operations can be performed on pointers such as assignment, dereferencing, pointer arithmetic, comparison, and passing pointers to functions.

Pointer Assignment

The address of one variable can be assigned to a pointer using the address operator &.

```
int a = 10;
```

```
int *p;
```

```
p = &a;
```

Here, p stores the address of a.

Program

```
#include <stdio.h>

int main()
{
    int a = 10;

    int *p;

    p = &a;

    printf("Value of a = %d\n", a);//10
    printf("Address of a = %d\n", &a);//
    printf("Value stored in p= %d\n", *p);//
    return 0;
}
```

Dereferencing a Pointer

The * operator is used to access the value stored at the address contained in a pointer.

```
int a = 10;

int *p = &a;

printf("%d", *p);
```

Here, *p gives the value of a.

Program

```
#include <stdio.h>

int main()
{
    int a = 25;

    int *p = &a;

    printf("Value of a = %d\n", a);

    printf("Value using pointer = %d\n", *p);

    return 0;
}
```

Pointer Arithmetic

Pointer arithmetic means performing arithmetic operations on pointer variables.

The main pointer arithmetic operations are:

- Increment ++
- Decrement --
- Addition +
- Subtraction -
- Difference between two pointers

Pointer arithmetic is commonly used with arrays.

Pointer Increment

When a pointer is incremented, it moves to the next memory location of its data type.

For example, if p is an integer pointer:

```
p++;
```

moves the pointer to the next integer.

Program

```
#include <stdio.h>

int main()
{
    int a[] = {10, 20, 30, 40};
    int *p = a;
    printf("Value = %d\n", *p);//10
    p++;
    printf("Value after increment = %d\n", *p);//20
    p++;
    printf("Value after increment = %d\n", *p);//30
    return 0;
}
```

Output

Value = 10

Value after increment = 20

Value after increment = 30

Pointer Decrement

The decrement operator -- moves a pointer to the previous element.

```
p--;
```

Program

```
#include <stdio.h>

int main()
{
    int a[] = {10, 20, 30, 40};
    int *p = &a[3];
    printf("Current value = %d\n", *p);
    p--;
    printf("After decrement = %d\n", *p);
    return 0;
}
```

Output

Current value = 40

After decrement = 30

Adding an Integer to a Pointer

An integer can be added to a pointer.

```
p + 2
```

moves the pointer two elements forward.

Program

```
#include <stdio.h>

int main()
{
    int a[] = {10, 20, 30, 40, 50};
    int *p = a;
    printf("Value at p = %d\n", *p);
    printf("Value at p + 2 = %d\n", *(p + 2));
    printf("Value at p + 4 = %d\n", *(p + 4));
    return 0;
}
```

```
}
```

Output

Value at p = 10

Value at p + 2 = 30

Value at p + 4 = 50

Subtracting an Integer from a Pointer

An integer can be subtracted from a pointer.

$p - 2$

moves the pointer two elements backward.

Program

```
#include <stdio.h>

int main()
{
    int a[] = {10, 20, 30, 40, 50};
    int *p = &a[4];
    printf("Value at p = %d\n", *p);
    printf("Value at p - 2 = %d\n", *(p - 2));
    return 0;
}
```

Output

Value at p = 50

Value at p - 2 = 30

Subtraction of Two Pointers

Two pointers pointing to elements of the same array can be subtracted.

The result gives the number of elements between them.

Program

```
#include <stdio.h>

int main()
{
    int a[] = {10, 20, 30, 40, 50};
    int *p1 = &a[1];
```

```
int *p2 = &a[4];  
printf("Difference = %td\n", p2 - p1);  
return 0;  
}
```

Output

Difference = 3

Comparison of Pointers

Pointers can be compared using relational operators such as:

==

!=

<

>

<=

>=

This is generally meaningful when comparing pointers into the same array.

Program

```
#include <stdio.h>  
  
int main()  
{  
    int a[] = {10, 20, 30};  
    int *p = &a[0];  
    int *q = &a[2];  
    if (p < q)  
        printf("p comes before q\n");  
    else  
        printf("p does not come before q\n");  
    return 0;  
}
```

Output

p comes before q

Passing Pointers to Functions

Passing the pointers to the function means the memory location of the variables is passed to the parameters in the function, and then the operations are performed. The function definition accepts these addresses using pointers, addresses are stored using pointers.

Arguments Passing without pointer

When we pass arguments without pointers the changes made by the function would be done to the local variables of the function. Below is the C program to pass arguments to function without a pointer:

```
// C program to swap two values
// without passing pointer to
// swap function.
#include <stdio.h>

void swap(int a, int b)
{
    int temp = a;
    a = b;
    b = temp;
}

// Driver code
int main()
{
    int a = 10, b = 20;
    swap(a, b);
    printf("Values after swap function are: %d, %d",
        a, b);
    return 0;
}
```

Output

Values after swap function are: 10, 20

Arguments Passing with pointers

A pointer to a function is passed in this example. As an argument, a pointer is passed instead of a variable and its address is passed instead of its value. As a result, any change made by the function

using the pointer is permanently stored at the address of the passed variable. In C, this is referred to as call by reference.

Below is the C program to pass arguments to function with pointers:

```
// C program to swap two values
// by passing pointers to the
// swap function.
```

```
#include <stdio.h>
```

```
void swap(int* a, int* b)
{
    int temp;
    temp = *a;
    *a = *b;
    *b = temp;
}
```

```
// Driver code
int main()
{
    int a = 10, b = 20;
    printf("Values before swap function are: %d, %d\n",
        a, b);
    swap(&a, &b);
    printf("Values after swap function are: %d, %d",
        a, b);
    return 0;
}
```

Output

Values before swap function are: 10, 20

Values after swap function are: 20, 10

Pointer to an Array

A pointer to an array is a pointer that points to the whole array instead of the first element of the array. It considers the whole array as a single unit instead of it being a collection of given elements.

Example:

```
#include<stdio.h>
```

```
int main() {  
    int arr[5] = { 1, 2, 3, 4, 5 };  
    int *ptr = arr;  
  
    printf("%p\n", ptr);  
    return 0;  
}
```

Output

0x7ffde273ac20

In the above program, we have a pointer *ptr* that points to the 0th element of the array. Similarly, we can also declare a pointer that can point to whole array instead of only one element of the array. This pointer is useful when talking about multidimensional arrays.

Syntax of Array Pointer

An array pointer can be declared as shown:

```
type(*ptr)[size];
```

where,

- type: Type of data that the array holds.
- ptr: Name of the pointer variable.
- size: Size of the array to which the pointer will point.

For example,

```
int (*ptr)[10];
```

Here *ptr* is pointer that points to an array of 10 integers. Since subscript have higher precedence than indirection, it is necessary to enclose the indirection operator and pointer name inside parentheses.

Examples of Pointer to Array

The following examples demonstrate the use of pointer to an array in C and also highlights the difference between the pointer to an array and pointer to the first element of an array.

Access Array Using Array Pointer

In the below code, the type of ptr is pointer to an array of 10 integers.

```
#include <stdio.h>
```

```
int main() {  
    int arr[3] = { 5, 10, 15 };  
    int n = sizeof(arr) / sizeof(arr[0]);  
  
    // Declare pointer variable  
    int (*ptr)[3];  
  
    // Assign address of val[0] to ptr.  
    // We can use ptr=&val[0];(both are same)  
    ptr = &arr;  
  
    for (int i = 0; i < n; i++)  
        printf("%d ", (*ptr)[i]);  
  
    return 0;  
}
```

Output

5 10 15

Find the Size of Array Passed to a Function

Normally, it is impossible to find the size of array inside a function, but if we pass the pointer to an array, then it is possible.

```
#include <stdio.h>
```

```
// Function that takes array as argument
```

```
void foo(int (*arr)[5]) {  
    printf("%lu ", sizeof(*arr));  
}
```

```

int main() {
    int arr[5];

    // Passing the address of arr
    foo(&arr);

    return 0;
}

```

Output

20

But again, we have to hardcode the size information.

Pointer to the First Element of Array vs Array Pointer

The pointer that points to the 0th element of array and the array pointer that points to the whole array are totally different. The following program shows this:

```
#include <stdio.h>
```

```

int main() {
    int arr[5] = {1, 2, 3, 4, 5};

    // Create a pointer to integer
    int *p;

    // Pointer to an array of 5 integers
    int(*ptr)[5];

    // Points to 0th element of the arr
    p = arr;

    // Points to the whole array arr
    ptr = &arr;
}

```

```

printf("p = %p\n", p);
printf("*ptr = %p\n\n", *ptr);

// incrementing both pointers

p++;
ptr++;

printf("p = %p\n", *p);
printf("*ptr = %p", ptr);

return 0;
}

```

Output

```

p = 0x7fff81e4caf0
*ptr = 0x7fff81e4caf0

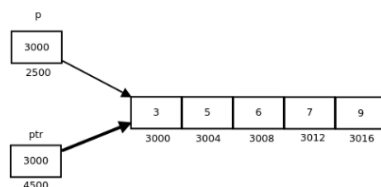
p = 0x7fff81e4caf4
*ptr = 0x7fff81e4cb04

```

Here, *p* is pointer to 0th element of the array *arr*, while *ptr* is a pointer that points to the whole array *arr*.

- The base type of *p* is *int* while base type of *ptr* is 'an array of 5 integers'.
- We know that the pointer arithmetic is performed relative to the base size, so if we write *ptr++*, then the pointer *ptr* will be shifted forward by 20 bytes.

The following figure shows the pointer *p* and *ptr*. The darker arrow denotes a pointer to an array. (address may vary in each execution of the program)



On dereferencing a pointer expression, we get a value pointed to by that pointer expression. The pointer to an array points to an array, so on dereferencing it, we should get the array, and the name of the array denotes the base address. So, whenever a pointer to an array is dereferenced, we get the base address of the array to which it points.

Pointer to Multidimensional Arrays

The concept of pointer to an array can be extended to multidimensional arrays too:

1. Pointers to 2D Arrays

To define a pointer to a 2D array, both the number of rows and columns of the array must be specified in the pointer declaration.

```
type *(ptr)[row][cols]
```

Let's take a look at an example:

```
#include <stdio.h>
```

```
int main() {  
    int arr[2][3] = {{1, 2, 3},  
                    {4, 5, 6}};  
  
    // pointer to above array  
    int (*ptr)[2][3] = &arr;  
  
    // Traversing the array using ptr  
    for (int i = 0; i < 2; i++) {  
        for (int j = 0; j < 3; j++) {  
            printf("%d ", (*ptr)[i][j]);  
        }  
        printf("\n");  
    }  
  
    return 0;  
}
```

Output

1 2 3

4 5 6

2. Pointers to 3D Arrays

To define a pointer to a 2D array, both the number of rows and columns of the array must be specified in the pointer declaration.

```
type *(ptr)[depth][row][cols]
```

Let's take a look at an example:

```
#include <stdio.h>
```

```
int main() {
```

```
    int arr[2][3][2] = {{{1, 2}, {3, 4}, {5, 6}},  
                        {{7, 8}, {9, 10}, {11, 12}}};
```

```
    // Pointer to the 3D array
```

```
    int (*ptr)[2][3][2] = &arr;
```

```
    // Traversing the 3D array using the pointer
```

```
    for (int i = 0; i < 2; i++) {
```

```
        for (int j = 0; j < 3; j++) {
```

```
            for (int k = 0; k < 2; k++) {
```

```
                printf("%d ", (*ptr)[i][j][k]);
```

```
            }
```

```
            printf("\n");
```

```
        }
```

```
        printf("\n");
```

```
    }
```

```
    return 0;
```

```
}
```

Output

1 2

3 4

5 6

7 8

9 10

11 12

Array of Pointers

In C, a pointer array is a homogeneous collection of indexed pointer variables that are references to a memory location. It is generally used in C Programming when we want to point at multiple memory locations of a similar data type in our C program. We can access the data by dereferencing the pointer pointing to it.

Syntax:

```
pointer_type *array_name [array_size];
```

Here,

- `pointer_type`: Type of data the pointer is pointing to.
- `array_name`: Name of the array of pointers.
- `array_size`: Size of the array of pointers.

Note: It is important to keep in mind the operator precedence and associativity in the array of pointers declarations of different type as a single change will mean the whole different thing. For example, enclosing `*array_name` in the parenthesis will mean that `array_name` is a pointer to an array.

Example:

```
// C program to demonstrate the use of array of pointers
```

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    // declaring some temp variables
```

```
    int var1 = 10;
```

```
    int var2 = 20;
```

```
    int var3 = 30;
```

```
    // array of pointers to integers
```

```
    int* ptr_arr[3] = { &var1, &var2, &var3 };
```

```
    // traversing using loop
```

```

for (int i = 0; i < 3; i++) {

    printf("Value of var%d: %d\tAddress: %p\n", i + 1, *ptr_arr[i], ptr_arr[i]);

}

return 0;

}

```

Output

Value of var1: 10 Address: 0x7fff1ac82484

Value of var2: 20 Address: 0x7fff1ac82488

Value of var3: 30 Address: 0x7fff1ac8248c

Explanation:



As shown in the above example, each element of the array is a pointer pointing to an integer. We can access the value of these integers by first selecting the array element and then dereferencing it to get the value.

Chain of Pointers

A chain of pointers is a series of pointers where each pointer stores the address of another pointer, creating multiple levels of indirection. It is commonly used to access or modify data through multiple pointer levels.

- A pointer stores the address of a variable, while a double pointer, triple pointer, and higher-level pointers store the addresses of other pointers.
- Chain of pointers is useful in dynamic memory management, passing pointers to functions, and working with complex data structures.

// level-1 pointer declaration

*datatype *pointer;*

// level-2 pointer declaration

*datatype **pointer;*

// level-3 pointer declaration

*datatype ***pointer;*

.

.

and so on

The level of the pointer depends on how many asterisks the pointer variable is preceded with at the time of declaration.

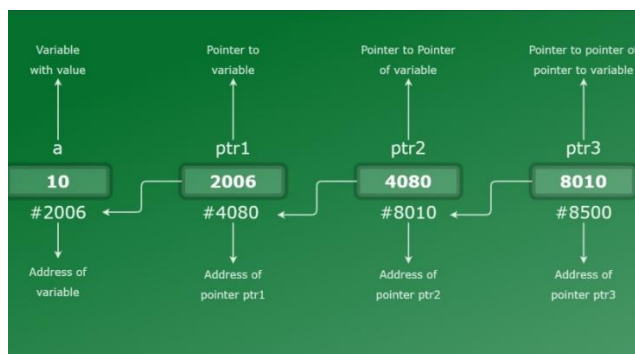
Declaration

```
int *pointer_1;  
int **pointer_2;  
int ***pointer_3;  
.  
.  
and so on
```

Level of pointers or say chain can go up to N level depending upon the memory size. If you want to create a pointer of level-5, you need to precede the pointer variable name by 5 asterisks(*) at the time of declaration.

Initialization

```
// initializing level-1 pointer with address of variable 'var'  
pointer_1 = &var;  
  
// initializing level-2 pointer with address of level-1 pointer  
pointer_2 = &pointer_1;  
  
// initializing level-3 pointer with address of level-2 pointer  
pointer_3 = &pointer_2;  
.  
.  
and so on
```



- Variable a stores the value 10, while ptr1, ptr2, and ptr3 successively store the addresses of a, ptr1, and ptr2.
- Since every pointer in the chain ultimately points to a, the value 10 can be accessed through any level of indirection.

```
#include <stdio.h>
```

```
// C program for chain of pointer
```

```
int main()
```

```

{
    int var = 10;

    // Pointer level-1
    // Declaring pointer to variable var
    int* ptr1;

    // Pointer level-2
    // Declaring pointer to pointer variable *ptr1
    int** ptr2;

    // Pointer level-3
    // Declaring pointer to double pointer **ptr2
    int*** ptr3;

    // Storing address of variable var
    // to pointer variable ptr1
    ptr1 = &var;

    // Storing address of pointer variable
    // ptr1 to level -2 pointer ptr2
    ptr2 = &ptr1;

    // Storing address of level-2 pointer
    // ptr2 to level-3 pointer ptr3
    ptr3 = &ptr2;

    // Displaying values
    printf("Value of variable "
        "var = %d\n",
        var);
    printf("Value of variable var using"

```

```

        " pointer ptr1 = %d\n",
        *ptr1);
printf("Value of variable var using"
        " pointer ptr2 = %d\n",
        **ptr2);
printf("Value of variable var using"
        " pointer ptr3 = %d\n",
        ***ptr3);

return 0;
}

```

Output

Value of variable var = 10

Value of variable var using pointer ptr1 = 10

Value of variable var using pointer ptr2 = 10

Value of variable var using pointer ptr3 = 10

Dynamic Memory Allocation in C

Dynamic memory allocation allows programs to allocate and manage memory at runtime according to their requirements. It is useful when the amount of memory needed by a program is not known at compile time.

- Dynamic memory is allocated from the heap.
- The allocated memory can be resized using `realloc()`.
- Dynamically allocated memory should be released using `free()` when it is no longer needed.

Dynamic Memory Allocation Functions

C provides four main functions for dynamic memory management: `malloc()`, `calloc()`, `realloc()`, and `free()`. These functions are declared in the `<stdlib.h>` header file.

`malloc()`

The `malloc()` function allocates a contiguous block of memory on the heap at runtime. The allocated memory is uninitialized, so its contents have an indeterminate value.

Syntax

```
ptr = malloc(size);
```

The size specifies the number of bytes to allocate, and `malloc()` returns a pointer to the allocated memory. If the allocation fails, it returns `NULL`.

Example: Suppose we want to dynamically allocate memory for 5 integers.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int main() {
```

```
    int *ptr = malloc(5 * sizeof(int));
```

```
    if (ptr == NULL) {
```

```
        printf("Memory allocation failed\n");
```

```
        return 1;
```

```
    }
```

```
    for (int i = 0; i < 5; i++) {
```

```
        ptr[i] = i + 1;
```

```
    }
```

```
    for (int i = 0; i < 5; i++) {
```

```
        printf("%d ", ptr[i]);
```

```
    }
```

```
    free(ptr);
```

```
    return 0;
```

```
}
```

Output

1 2 3 4 5

Explanation: `malloc(5 * sizeof(int))` allocates enough memory to store 5 integers, while `ptr` stores the address of the allocated memory. The allocated memory is used like an array, and `free(ptr)` releases it after use.

Checking `malloc()` Failure

Memory allocation may fail if the requested amount of memory is unavailable. Therefore, the returned pointer should be checked against NULL before using the allocated memory.

```
#include <stdio.h>
```

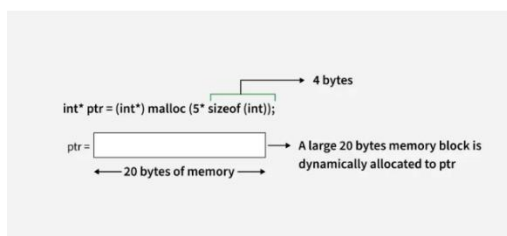
```
#include <stdlib.h>
```

```
int main() {  
    int *ptr = malloc(5 * sizeof(int));  
  
    if (ptr == NULL) {  
        printf("Memory allocation failed\n");  
        return 1;  
    }  
  
    printf("Memory allocated successfully\n");  
  
    free(ptr);  
  
    return 0;  
}
```

Output

Memory allocated successfully

*Note: In C, an explicit cast such as (int *)malloc(...) is not required. malloc() returns a void *, which can be implicitly converted to another object pointer type in C.*



This function returns a void pointer to the allocated memory that needs to be converted to the pointer of required type to be usable. If allocation fails, it returns NULL pointer.

calloc()

The `calloc()` function allocates memory for multiple elements and initializes all allocated bytes to zero.

Syntax

```
ptr = calloc(number_of_elements, size_of_each_element);
```

Unlike `malloc()`, `calloc()` accepts the number of elements and the size of each element separately.

```
#include <stdio.h>
```

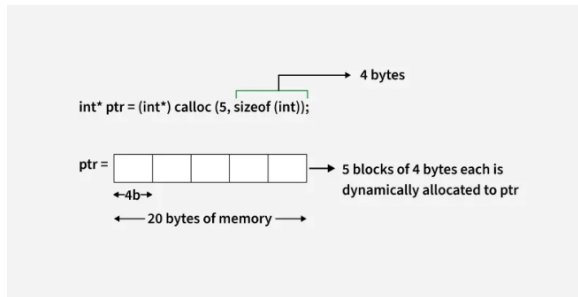
```
#include <stdlib.h>
```

```
int main() {  
    int *ptr = calloc(5, sizeof(int));  
  
    if (ptr == NULL) {  
        printf("Memory allocation failed\n");  
        return 1;  
    }  
  
    for (int i = 0; i < 5; i++) {  
        printf("%d ", ptr[i]);  
    }  
  
    free(ptr);  
  
    return 0;  
}
```

Output

0 0 0 0 0

Explanation: `calloc(5, sizeof(int))` allocates memory for 5 integers and initializes the allocated bytes to zero. The pointer `ptr` can then be used to access the allocated memory like an array.



This function also returns a void pointer to the allocated memory that is converted to the pointer of required type to be usable. If allocation fails, it returns NULL pointer.

`free()`

The `free()` function releases dynamically allocated memory that is no longer required. Memory allocated using `malloc()`, `calloc()`, or `realloc()` should eventually be released using `free()`.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int main() {
```

```
    int *ptr = calloc(5, sizeof(int));
```

```
    if (ptr == NULL) {
```

```
        printf("Memory allocation failed\n");
```

```
        return 1;
```

```
    }
```

```
    for (int i = 0; i < 5; i++) {
```

```
        printf("%d ", ptr[i]);
```

```
    }
```

```

    free(ptr);

    ptr = NULL;

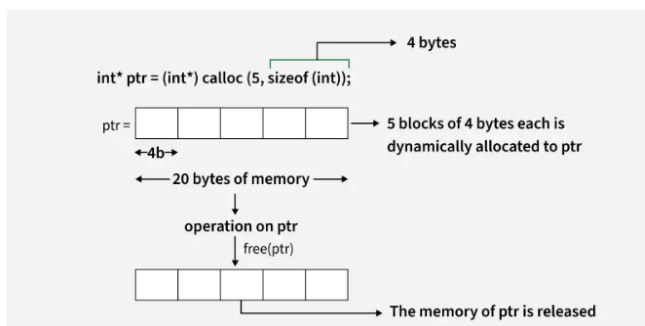
    return 0;
}

```

Output

0 0 0 0 0

Explanation: `free(ptr)` releases the memory allocated for `ptr`, making that memory available for future use. Setting `ptr` to `NULL` after `free()` helps prevent accidental use of the invalid pointer.



After freeing a memory block, the pointer becomes invalid, and it is no longer pointing to a valid memory location.

`realloc()`

The `realloc()` function changes the size of a previously allocated memory block. It can be used to increase or decrease the amount of dynamically allocated memory.

Syntax

```
ptr = realloc(ptr, new_size);
```

If the memory block is successfully resized, `realloc()` returns a pointer to the resized block. If it fails, it returns `NULL` and the original memory block remains allocated.

Example: Suppose we initially allocate memory for 5 integers and later need space for 10 integers.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int main() {
```

```
    int *ptr = malloc(5 * sizeof(int));
```

```
    if (ptr == NULL) {
```

```
    printf("Memory allocation failed\n");
    return 1;
}

for (int i = 0; i < 5; i++) {
    ptr[i] = i + 1;
}

int *temp = realloc(ptr, 10 * sizeof(int));

if (temp == NULL) {
    printf("Memory reallocation failed\n");
    free(ptr);
    return 1;
}

ptr = temp;

for (int i = 5; i < 10; i++) {
    ptr[i] = i + 1;
}

for (int i = 0; i < 10; i++) {
    printf("%d ", ptr[i]);
}

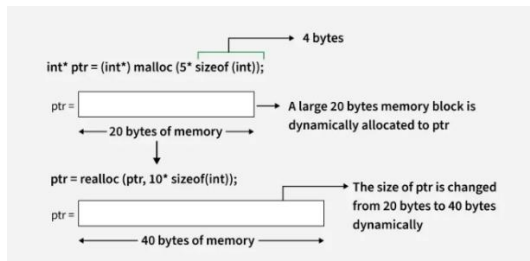
free(ptr);

return 0;
}
```

Output

1 2 3 4 5 6 7 8 9 10

Explanation: The program initially allocates memory for 5 integers and then uses `realloc()` to resize the block to hold 10 integers. A temporary pointer is used to safely handle reallocation failure without losing the original memory block.



It is important to note that if `realloc()` fails and returns `NULL`, the original memory block is not freed, so you should not overwrite the original pointer until you've successfully allocated a new block. To prevent memory leaks, it's a good practice to handle the `NULL` return value carefully:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int main()
```

```
{
```

```
    int *ptr = (int *)malloc(5 * sizeof(int));
```

```
    // Reallocation
```

```
    int *temp = (int *)realloc(ptr, 10 * sizeof(int));
```

```
    // Only update the pointer if reallocation is successful
```

```
    if (temp == NULL)
```

```
        printf("Memory Reallocation Failed\n");
```

```
    else
```

```
        ptr = temp;
```

```
    return 0;
```

```
}
```

User-Defined Data Types In C

Data Types are the types of data that can be stored in memory using a programming language. Basically, data types are used to indicate the type of data that a variable can store. These data types require different amounts of memory and there are particular operations that can be performed on them. These data types can be broadly classified into three types:

1. Primitive Data Types
2. Derived Types
3. User Defined Data Types

Types of User-Defined DataTypes

There are 4 types of user-defined data types in C. They are

Structure

Union

Enum

Typedef

Structures C

A structure in C is a user-defined data type that groups related variables of different data types under a single name. It is useful for representing real-world entities and organizing related data into a single unit.

- The struct keyword is used to define a structure, and its variables are called members.
- Structure members can be of different valid data types, including arrays, pointers and other structures.
- Structures are widely used to represent entities such as students, employees and products, and to build data structures such as linked lists and trees.

```
#include <stdio.h>
```

```
// Defining a structure
```

```
struct A {
```

```
    int x;
```

```
};
```

```
int main() {
```

```
    // Creating a structure variable
```

```

struct A a;

// Initializing member
a.x = 11;

printf("%d", a.x);

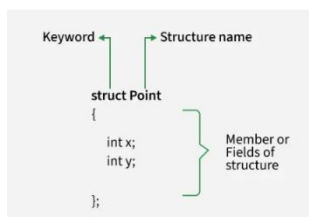
return 0;
}

```

Output

11

Explanation: In the above example, a structure A is defined to hold an integer member x. A variable a of type struct A is created and its member x is initialized to 11 by accessing it using dot operator. The value of a.x is then printed to the console.



Basic Operations of Structure

Following are the basic operations commonly used on structures:

1. Access Structure Members

- To access or modify members of a structure, we use the (.) dot operator. This is applicable when we are using structure variables directly.
- In the case where we have a pointer to the structure, we can also use the arrow operator to access the members.

```
#include <stdio.h>
```

```

struct Student {
    int age;
    float marks;
};

```

```
int main() {
```

```

struct Student s = {20, 85.5};

// Accessing members using dot operator
printf("Age: %d\n", s.age);
printf("Marks: %.1f\n", s.marks);

// Creating a pointer to the structure
struct Student *ptr = &s;

// Accessing members using arrow operator
printf("Age using pointer: %d\n", ptr->age);
printf("Marks using pointer: %.1f", ptr->marks);

return 0;
}

```

Output

Age: 20

Marks: 85.5

Age using pointer: 20

Marks using pointer: 85.5

Explanation: In above example, s is a structure variable, while ptr stores its address. The code accesses the same age and marks members through both s and ptr and prints their values.

2. Initialize Structure Members

Structure members cannot be initialized inside the structure definition. However, they can be initialized at the time of declaring a structure variable using initializer lists.

```
#include <stdio.h>
```

```
// Defining a structure to represent a student
```

```
struct Student
```

```
{
```

```

char name[50];

int age;

float grade;
};

int main()
{

    // Declaring and initializing a structure variable
    struct Student s1 = {"Rahul", 20, 18.5};

    // Designated Initializing another structure
    struct Student s2 = {.age = 18, .name = "Vikas", .grade = 22};

    // Accessing structure members
    printf("%s\t%d\t%.2f\n", s1.name, s1.age, s1.grade);
    printf("%s\t%d\t%.2f\n", s2.name, s2.age, s2.grade);

    return 0;
}

```

Output

```

Rahul  20    18.50
Vikas  18    22.00

```

Explanation: In the above example, the Student structure contains name, age, and grade members. s1 is initialized using an initializer list, while s2 uses designated initializers to assign values to specific members. The values of both structure variables are then accessed and printed using the dot (.) operator.

4. Passing Structure to Functions

Structure can be passed to a function in the same way as normal variables. Though, it is recommended to pass it as a pointer to avoid copying a large amount of data.

```
#include <stdio.h>
```



```

// Structure definition
struct A {
    int x;
};

// Function to increment values
void increment(struct A a, struct A* b) {
    a.x++;
    b->x++;
}

int main() {
    struct A a = { 10 };
    struct A b = { 10 };

    // Passing a by value and b by pointer
    increment(a, &b);

    printf("a.x: %d \tb.x: %d", a.x, b.x);
    return 0;
}

```

Output

a.x: 10 b.x: 11

Explanation: In this example, a is passed to the increment() function by value, while b is passed by pointer. The change to a.x affects only its copy, whereas b->x modifies the original structure.

Self-Referential Structures

A self-referential structures contains a pointer member that points to another object of the same structure type. Self-referential structures are commonly used to implement dynamic data structures such as linked lists, trees and graphs.

```

struct Node {
    int data;

```

```
struct Node *next;
```

```
};
```

Unions in C

A union is a user-defined data type that allows storing different data types in the same memory location. At any given time, only one member can hold a valid value because all members share the same memory.

- All union members share the same memory location, so updating one member overwrites the others.
- Unions are useful for saving memory when only one of several data members is needed at a time.

```
#include <stdio.h>
```

```
union Student {
```

```
    int rollNo;
```

```
    float height;
```

```
    char firstLetter;
```

```
};
```

```
int main() {
```

```
    // Declare a union variable
```

```
    union Student data;
```

```
    // Assign and print the roll number
```

```
    data.rollNo = 21;
```

```
    printf("%d\n", data.rollNo);
```

```
    data.height = 5.2;
```

```
    printf("%.2f\n", data.height);
```

```
    data.firstLetter = 'N';
```

```
    printf("%c", data.firstLetter);
```

```
    return 0;
```

```
}
```

Output

21

5.20

N

Union Declaration

A union is declared similarly to a structure. Provide the name of the union and define its member variables.

```
union UnionName {  
    data_type member1;  
    data_type member2;  
    ...  
};
```